Project - High Level Design

On

Healthcare Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Chetan Oswal	EN22CS301295
2	Irya Patni	EN22CS301436
3	Uday Dubey	EN22EL301058
4	Himani Jaiswal	EN22CS301423

Group Name: Placed BtechGroup07

Project Number: AAI-02

Industry Mentor Name:

University Mentor Name :

Academic Year: 2025-2026

Table of Contents

1. Introduction 3
2. Scope of the Document 3
3. Intended Audience 3
4. System Overview 4
5. System Design 4
6. Application Design 5
7. Process Flow 6
8. Information Flow 7
9. Components Design 7
10. Key Design Considerations 8
11. API Catalogue 8
12. Data Design 9
13. Data Model 9
14. Data Access Mechanism 9
15. Data Retention Policies 10
16. Data Migration 10
17. Interfaces 10
18. State and Session Management 10
19. Caching 10
20. Non-Functional Requirements 11
21. Security Aspects 11
22. Performance Aspects 11
23. References 11

1. Introduction

Planning and managing medical resources efficiently can be challenging because hospitals must coordinate doctors, equipment, and other resources while responding to patient needs in real time. Manual planning methods often lead to delays, scheduling conflicts, and inefficient use of available resources.

To address this issue, we developed the Healthcare Planning Assistant Agent, an intelligent system that assists in generating optimized medical planning and scheduling. The system uses artificial intelligence to analyze requests, reason about available resources, and suggest suitable plans. By integrating AI reasoning with real-time resource information, the system helps automate parts of the planning process.

The main aim of this project is to improve the efficiency of medical resource planning and support healthcare staff in making faster and more informed scheduling decisions.

2. Scope of the Document

This document explains the overall design of our system. It includes:

- How the system works
- The architecture and components
- Data handling methods
- APIs used
- Security and performance considerations

This document gives a high-level understanding of our project structure.

3. Intended Audience

This document is prepared for:

- Faculty members and evaluators
- Industry mentor
- Developers working on the project
- Anyone who wants to understand the system design

4. System Overview

The Healthcare Planning Assistant Agent is an AI-assisted planning system designed to help generate medical schedules and resource plans based on user requests.

1. The user enters a medical planning request through the interface.
2. The system processes the request using a backend server and an AI planning agent.
3. The AI agent analyzes the request and consults an AI language model to reason about the best possible plan.
4. The system also checks available hospital resources through a resource availability tool based on the reasoning and resource information, the system generates an optimized medical schedule or plan.
5. The final result is then displayed to the user through the interface.

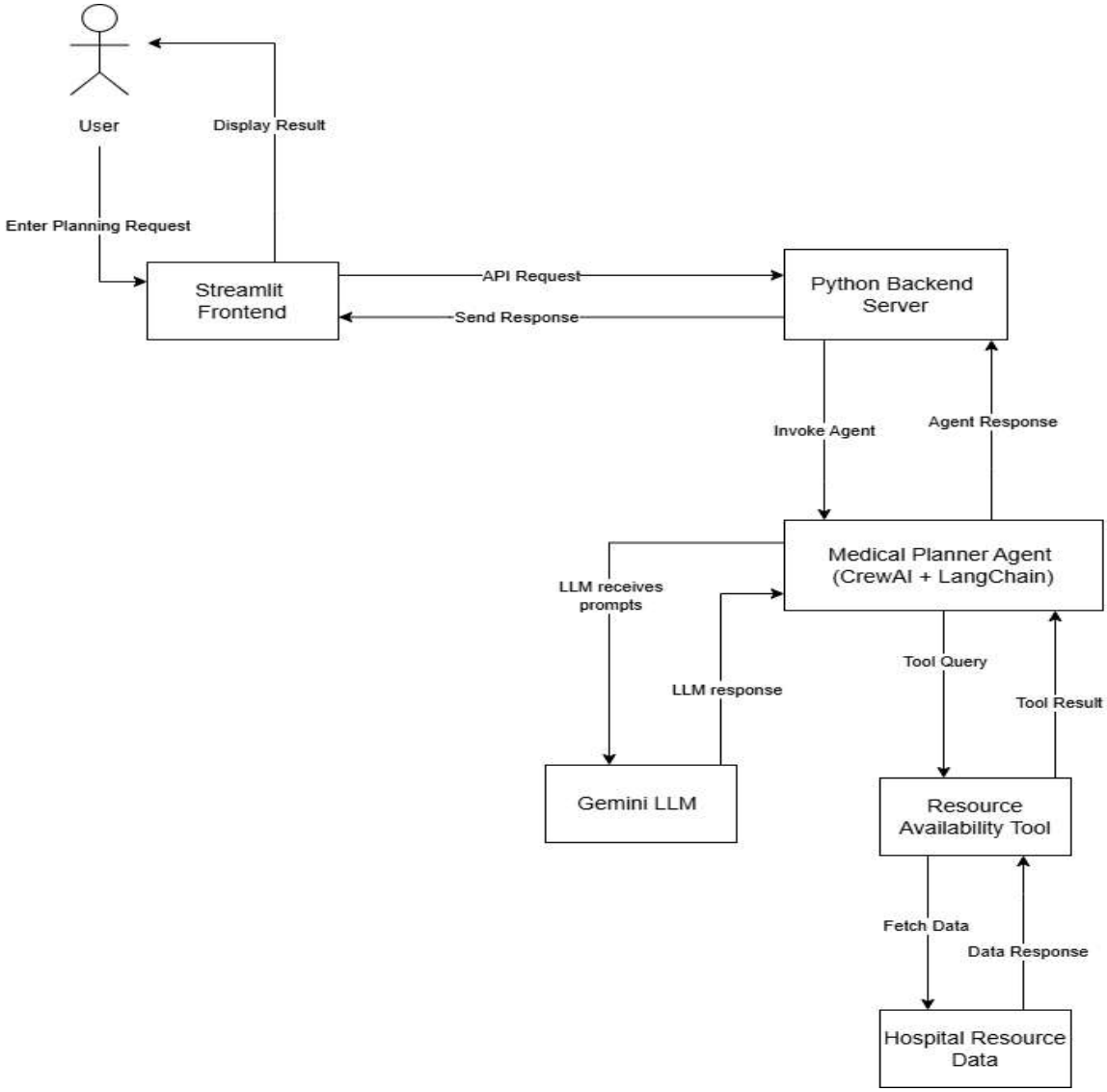
This automated approach helps simplify the planning process and improves the efficiency of managing medical resources.

5. System Design

Our system follows a modular design approach. It consists of:

- User Interface
- Backend Processing System
- Medical Planning Agent Module
- Resource Availability Module
- Output Display Module

Each part performs a specific task, and together they deliver the final result.



6. Application Design

The application is designed as a simple web-based system.

It includes:

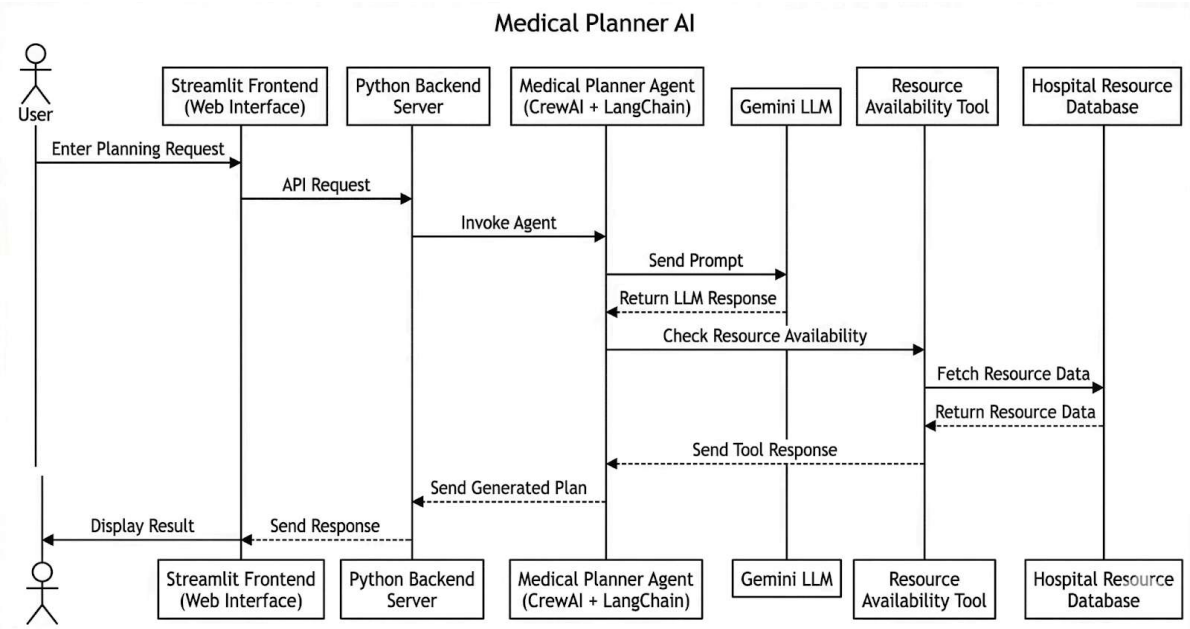
- A user interface built using Streamlit where users can enter their medical planning request.
- A backend system built using Python to handle processing and system logic
- Integration with the Gemini API to support AI reasoning and planning
- A resource availability module that retrieves hospital resource information.

Structured prompts and agent-based reasoning ensure relevant medical plan generation.

7. Process Flow

The process works in the following steps:

1. The user enters a medical planning request through the interface.
2. The backend system receives the request and processes the input
3. The request is passed to the medical planning agent for reasoning.
4. The agent sends a prompt to the Gemini API to analyze the request and generate planning logic.
5. The system checks available hospital resources through the resource availability tool.
6. Based on the reasoning and resource information, the system generates the final medical schedule or plan.
7. The final result is then displayed to the user through the interface.

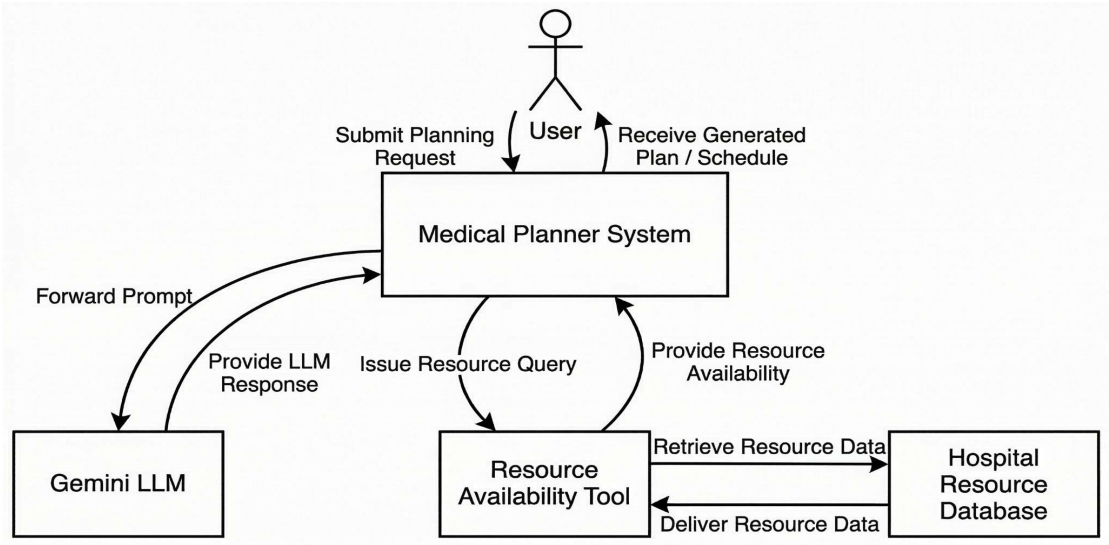


8. Information Flow

The information flows as follows:

User → Frontend Interface → Backend Server → Medical Planning Agent
Medical Planning Agent → Gemini AI Model → Reasoning Output
Medical Planning Agent → Resource Availability Tool → Hospital Resource Data

The agent processes AI reasoning and resource information to generate the final medical plan. This plan is sent to the backend server, which forwards it to the frontend for user display.



9. Components Design

1. User Interface

Enables user input of medical planning requests and display of results.

2. Backend Module

Manages user requests and system component communication.

3. Medical Planning Agent Module

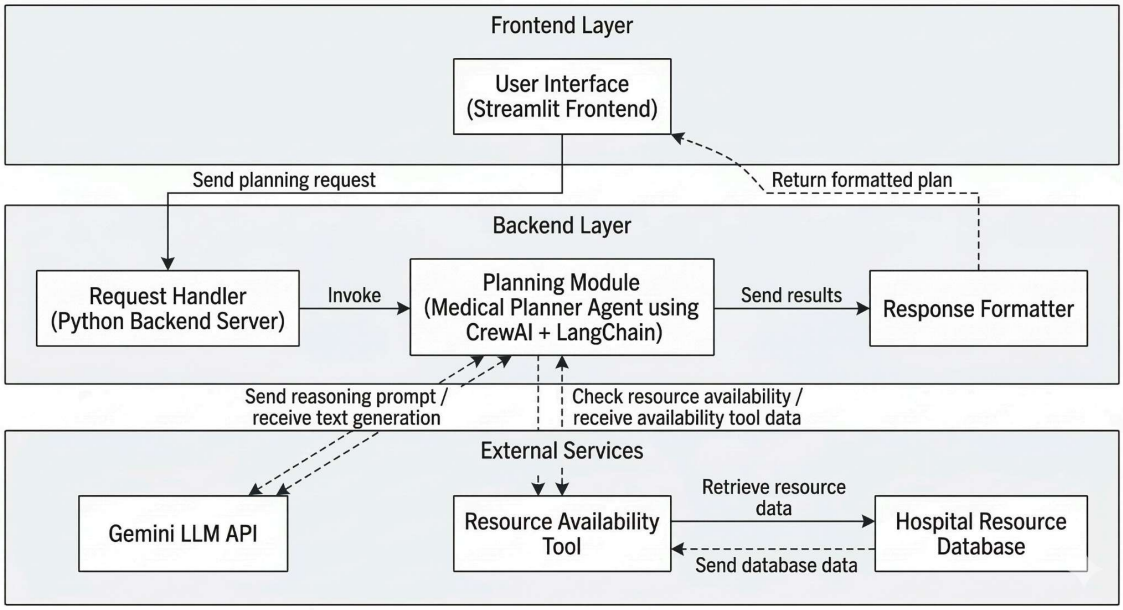
Uses the Gemini API to analyze requests and generate medical schedules or plans.

4. Resource Availability Module

Assesses hospital resources (doctors, equipment, facilities) needed for planning.

5. Output Module

Formats the generated plan for final user display.



10. Key Design Considerations

While designing the system, we focused on:

- Keeping the interface simple and easy to use
- Ensuring generated medical plans are clear and relevant to the request.
- AI reasoning and resource data will improve planning.
- Design the system for future scalability in terms of users and data.
- Modular architecture supports easy addition of future features or modules.

11. API Catalogue

Gemini LLM API

- Used for AI reasoning and planning logic generation
- Takes the planning request as input
- Returns reasoning output used to generate the medical plan

Resource Availability Module

- Used for checking hospital resource availability
- Takes resource queries as input
- Returns information about available doctors, equipment, or facilities

12. Data Design

The system mainly works with dynamic data.

Types of data used:

- User planning requests
- AI reasoning outputs
- Resource availability data
- Generated medical schedules or plans

The system does not store large amounts of persistent user data.

13. Data Model

Basic structure:

User

- User ID
- Planning Request

Response

- Response ID
- Generated Plan
- Resource Information
- Timestamp

14. Data Access Mechanism

- Communication between components is handled through APIs.
- Data is transferred using JSON format
- Secure API keys are used for authenticating requests to external services.

15. Data Retention Policies

- User planning requests are not permanently stored.
- Temporary session data is used only during request processing.
- The system does not store sensitive patient or personal data.

16. Data Migration

Currently, data migration is not required because the system does not rely on a permanent database.

In the future, if the system is deployed in hospitals or healthcare platforms, cloud storage or database integration may be considered.

17. Interfaces

External Interfaces

- Gemini API
- Resource Availability Module

Internal Interfaces

- Backend to AI agent communication
- Backend to user interface communication

18. State and Session Management

- Each user request is processed independently.
- The system follows a stateless backend design.
- Sessions exist only during the processing of a request.

19. Caching

- Temporary caching can reduce repeated AI or resource requests.
- This helps improve system response time and efficiency.

20. Non-Functional Requirements

The system should:

- Respond quickly to planning requests
- Be reliable and stable
- Provide a simple and user-friendly interface
- Handle multiple user requests efficiently
- Maintain high system availability

21. Security Aspects

- API keys are stored securely in the backend environment.
- Communication with external services uses secure HTTPS protocols.
- Input validation is applied to prevent invalid or malicious requests.
- No sensitive medical or personal data is permanently stored.

22. Performance Aspects

- Optimized prompts for efficient AI reasoning
- Efficient backend processing using modular components
- Parallel handling of reasoning and resource checks
- Reduced response time through caching and optimized workflows

23. References

- Gemini API Documentation
- Python Official Documentation
- Streamlit Documentation
- Research on AI Agents and Prompt Engineering